Coding & Solution

CrimeVision — Intelligent Multimodal Forensic Analysis Platform

Date	1 November 2026
Team ID	[Enter Team ID]
Project Name	CrimeVision
Maximum Marks	5 Marks

Coding & Solution

CrimeVision is a Flask-based prototype that organizes digital evidence by case and provides supported media-analysis workflows. The implementation uses Python on the backend and HTML, CSS, and JavaScript for the browser interface.

Instructions / Submission Contents

Include the application source, dependency list, configuration template, and setup instructions. Keep secrets such as API keys out of the repository. Provide a repository link only if the project has actually been pushed to a hosted repository.

Solution Summary

Field	Details
Repository Link / URL	[Add repository URL if available; otherwise write Not published]
Programming Language(s)	Python, JavaScript, HTML, CSS
Framework(s) Used	Flask, Flask-CORS
Key Features Implemented	Case management; evidence registration; supported image/object detection; saved case/analysis records; API endpoints; sample YOLO object-detection run verified.
Pending / Incomplete Features	Confirm end-to-end audio transcription and video workflows; complete evidence preview/reopen flow if not yet tested; authentication, audit logging, and production-grade chain-of-custody controls are not confirmed.
Setup / Run Instructions	1. Open WSL Ubuntu and go to ~/CrimeVision. 2. Activate the virtual environment: source .venv/bin/activate. 3. Install dependencies if needed: pip install -r requirements.txt. 4. Ensure required .env values are configured without sharing secrets. 5. Run the project's configured entry point (for example, python run.py if that is the entry point). 6. Open the local URL printed by Flask.

Code Quality Checklist

S.No	Criteria	Status	Remarks
1	Code is modular and organized into functions / modules	Yes	Flask blueprints, case-store services, and upload utilities separate responsibilities.
2	Meaningful variable and function names are used	Yes	Route and helper names generally describe their purpose.
3	Comments / documentation are present where necessary	Partial	Module docstrings exist; consistency should be checked before handover.
4	Error handling is implemented for critical operations	Partial	Upload validation and HTTP errors exist; broad edge-case testing remains necessary.
5	Application runs without critical errors	Partial	Environment checks passed and a sample object-detection run succeeded; verify the full demo workflow.
6	Code is committed to a version control repository	Not confirmed	Only mark Yes if a Git repository with the submitted code is available.

Additional Notes / Comments

Current environment: WSL 2 Ubuntu, Python virtual environment, CPU-only PyTorch. The prototype supports initial review and organization of evidence; outputs require human verification. Do not claim full multimodal fusion or production forensic compliance unless implemented and validated.